UNIVERSIDADE FEDERAL DO PIAUÍ
CENTRO DE CIÊNCIAS DA NATUREZA–CCN
CAMPUS MINISTRO PETRÔNIO PORTELLA
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

RELATÓRIO DO SEXTO TRABALHO DE ESTRUTURA DE DADOS

DISCENTE GUSTAVO LIMA RESENDE

DISCENTE ARTHUR ANDRADE LIMA

DISCENTE THALISSON BRENO GOMES DE OLIVEIRA

TERESINA
OUTUBRO/2026

1. Introdução

O programa foi feito em python, e está dividido em 3 arquivos: a classe Node, a classe HashTable, e o main. A classe **Node** dá origem às listas encadeadas que serão implementadas na tabela hash. A classe **HashTable** corresponde à tabela hash e seus métodos. O **main** possui a função de mostrar os gráficos, além da leitura de arquivos.

2. Leitura dos arquivos TXT

O arquivo de alunos é lido linha a linha. Cada linha, que corresponde a um nome, é armazenada no array "alunos". Já o arquivo de Clara dos Anjos é inteiramente armazenado na variável "texto", para depois ser realizado o processamento de texto com a biblioteca "re". As palavras processadas são armazenadas no array "palavras_processadas". Também é evitada a repetição de palavras nesse array.

3. Hash Map

O hashMap é feito baseado no descrito em sala de aula, sendo implementado no array de listas encadeadas **self.table**. Os atributos **M** e **N** correspondem a, respectivamente, a quantidade de posições do array e a quantidade total de elementos no HashMap. O self.quant[] armazena a quantidade de elementos em cada índice do do hashMap;

O método insert() insere elementos no HashMap. Primeiramente, ele cria o Node com o nome recebido, e depois recebe, por meio de um número de 1 a 3, a função hash que será utilizada para armazenar o item. Se o índice em que o item vai ser armazenado estiver vazio, ele será inserido imediatamente. Caso contrário, ele será armazenado como o próximo item de uma lista encadeada.

Após ser armazenado, o contador de elementos N será incrementado em 1, e o self.quant[] terá um incremento no índice em que o elemento foi armazenado.

A verificação de elementos repetidos ocorre antes da chamada de inserção no hashmap, ainda no array, na função main.

4. Funções Hash

HashFirstL: função hash baseada na primeira letra da palavra. Ela pega o código ASCII da primeira letra e subtrai pelo da letra 'a', para obter um valor entra 0 e 25;

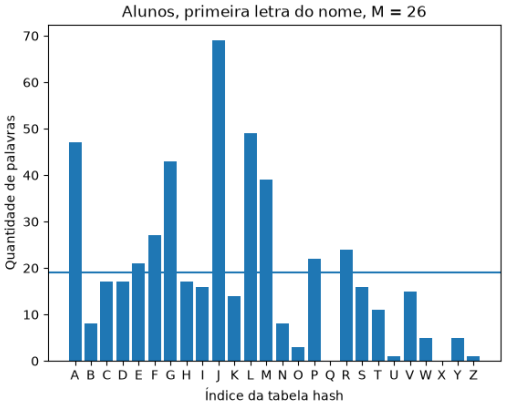
Hash_nativo: função hash nativa do Python;

_hash: função hash estudada em sala de aula, baseada em todas as letras da palavra;

5. Gráficos (Gráficos feitos utilizando matplotlib, e salvos em png)

A função MostrarGraficos() recebe o array de palavras, o tamanho M da tabela Hash, o número idx_hash que representa a função hash a ser utilizada, e o título do gráfico. Então os nomes do array são armazenados em uma tabela hash e os gráficos são criados com base na quantidade armazenada em cada índice da tabela

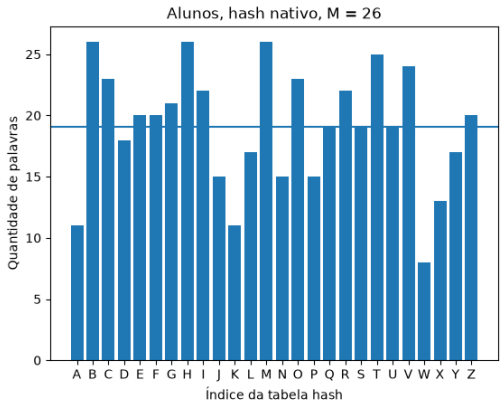
Histograma pedido na 1° questão: alunosED2026.txt, M = 26, 1° letra do nome como função hash:



Mais ocorrências: J (69)

Menos ocorrências: Q, X (0)

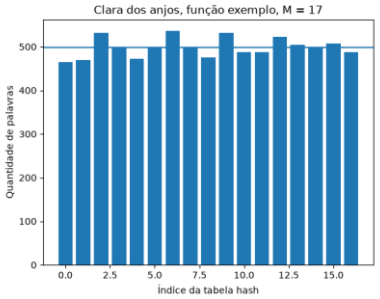
Histograma pedido na 2° questão: alunosED2026.txt, M = 26, função hash nativa da linguagem:

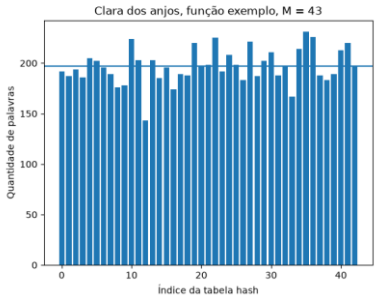


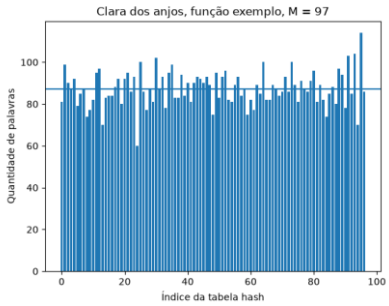
Mais ocorrências: B, H, M (26)

Menos ocorrências: W (8)

Histogramas pedidos na 3° questão: clara-dos-anjos.txt, M = 17, 43, 97, função hash exemplo em sala de aula:

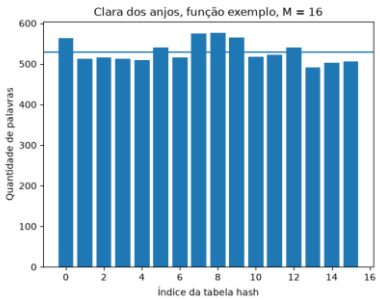


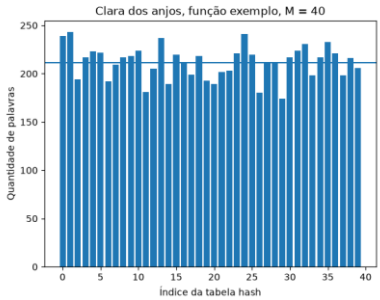


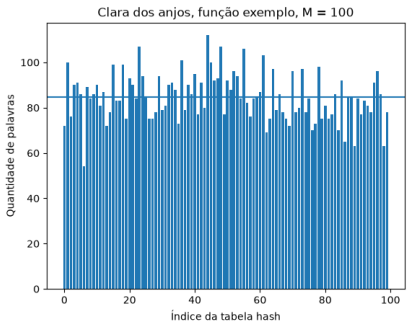


A tabela Hash com M = 17 possui uma distribuição mais uniforme, porém, com uma grande quantidade de colisões. Já M = 97 possui uma distribuição também relativamente uniforme, porém, dividido entre mais índices, o que reduz bastante a concentração em cada índice.

Histogramas pedidos na 4° questão: clara-dos-anjos.txt, M = 16, 40, 100, função hash exemplo em sala:







Para M = 16/17, e M = 40/43, os valores pares possuem uma oscilação bem semelhante no histograma, com os valores pares sendo ligeiramente superiores. Porém, para m = 100/97, é visível que com M = 100, os valores possuem bastantes oscilações, enquanto com M = 97 é ligeiramente mais estável. Portanto, nesse caso, o número primo foi uma opção melhor. Mas, nesse teste, no geral, os primos não foram muito superiores.